

Packet Tracer - Implementar API REST con un controlador SDN

Tabla de direccionamiento

Nota: Todas las máscaras de subredes son /24 (255.255.255.0).

Dispositivo	Interfaz	Dirección IP
R1	G0/0/0	192.168.101.1/24
	S0/1/0	192.168.1.2
R2	G0/0/0	192.168.102.1
	S0/1/1	192.168.2.2
R3	G0/0/0	10.0.1.1
	G0/0/1	10.0.2.1
	S0/1/0	192.168.1.1
	/1/1	192.168.2.1
SWL1	VLAN 1	192.168.101.2
SWL2	VLAN 1	192.168.102.2
SWR1	VLAN 1	10.0.1.2
SWR2	VLAN 1	10.0.1.3
SWR3	VLAN 1	10.0.1.4
SWR4	VLAN 1	10.0.1.5
Administrador	NIC	10.0.1.129
PC1	NIC	10.0.1.130
PC2	NIC	10.0.2.129
PC3	NIC	10.0.2.130
PC4	NIC	192.168.102.3
Ejemplo de Servidor	NIC	192.168.101.100
Controlador PT	NIC	192.168.101.254

Objetivos

Parte 1: Ejecutar la máquina virtual (Virtual Machine) de DEVASC.

Parte 2: Verificar la conectividad externa con el Packet Tracer

Parte 3: Solicitar un token de autenticación con Postman

Parte 4: Enviar solicitudes REST con Postman

Parte 5: Enviar solicitudes REST con código VS

Parte 6: Enviar solicitudes REST dentro del Packet Tracer

Aspectos básicos/Situación

En esta actividad de Packet Tracer, utilizará el controlador de red de Packet Tracer y la documentación de API asociada para enviar solicitudes REST desde Postman y desde código de Visual Studio (código VS). Packet Tracer también admite un entorno de codificación Python. Por lo tanto, en la parte final de esta actividad, enviará solicitudes REST desde dentro de Packet Tracer.

Recursos necesarios

- Una computadora con el sistema operativo de su elección.
- VirtualBox o VMware.
- Máquina virtual (Virtual Machine) DEVASC.

Instrucciones

Parte 1: Ejecute la máquina virtual (virtual machine) de DEVASC.

Si aún no ha completado el **Laboratorio: Instale el Entorno de Laboratorio de Máquina Virtual**, ahora. Si ya ha completado ese laboratorio, ejecute la máquina virtual (Virtual Machine) de DEVASC.

Parte 2: Verificar la conectividad externa con el Packet Tracer

En esta parte, verificará que otras aplicaciones de la VM DEVASC puedan acceder a Packet Tracer. Esta actividad debe completarse completamente dentro del entorno de la máquina virtual DEVASC. No se proporciona soporte para otras configuraciones.

Paso 1: Si aún no lo ha hecho, abra la actividad de Packet Tracer.

- a. En la máquina virtual de DEVASC, acceda a su plan de estudios en el navegador Chromium.
- b. Vaya a la página de esta actividad.
- c. Descargue e inicie el archivo **Packet Tracer - Implementar API REST con un SDN Controller.pka** asociado a estas instrucciones.

Paso 2: Verifique la configuración de Packet Tracer para el acceso externo.

- a. Haga clic en **Opciones > Preferencias > Micellaneous**. En **Acceso a red externa**, compruebe que la opción **Habilitar acceso externo para la API REST del controlador de red** está activada.
- b. Cierre la ventana de **Preferencias**.
- c. Haga clic en **PT-Controller0 > Config**.
- d. A la izquierda, en **MUNDO REAL**, haga clic en **Controller**.
- e. Marque el **Access Enabled** y anote el número de puerto, que probablemente debe ser **58000**. Este es el número de puerto que necesitará cuando acceda externamente a la actividad de Packet Tracer desde Chromium, VS Code y Postman más adelante en esta actividad.

Paso 3: Compruebe que puede acceder a Packet Tracer desde otro programa en la VM DEVASC.

Abre Chromium y navega hasta <http://localhost:58000/api/v1/host>.

Obtendrá la siguiente respuesta. Este paso verifica que puede acceder externamente a Packet Tracer y **PT-controller0**. Observe que la autorización requiere un ticket. Obtendrá un token de autorización en la siguiente Parte.

```
{
  "response": {
    «detail»: «Security Authentication Failure»,
    «ErrorCode»: «REST_API_EXTERNAL_ACCESS»,
    "message": "Ticket-based authorization: empty ticket."
  },
  "version": "1.0"
} {
```

Parte 3: Solicitar un token de autenticación con Postman

En esta parte, investigará la documentación de la API REST en Packet Tracer y utilizará Postman para solicitar un token de autenticación desde **PT-Controller0**. También puede hacer esto en VS Code con un script de Python.

Paso 1: Investigue la documentación de la API REST para el controlador de red.

Para ver la documentación de la API REST para **PT-Controller0**, siga los pasos siguientes:

- Haga clic en **Admin > Desktop > Web Browser**.
- Escriba **192.168.101.254**.
- Inicie sesión en **PT-Controller0** con el usuario **cisco** y la contraseña **cisco123!**.
- Haga clic en el menú situado junto al logotipo de Cisco y elija **API Docs**.
- También puede acceder a esta misma documentación desde el menú Ayuda. Haga clic en **Help > Contents**.
- En el panel de navegación de la izquierda, desplácese hacia abajo alrededor de dos tercios del camino y haga clic en **API de controlador de red**. Esto proporciona la misma documentación que encontró en **PT-Controller0**.
- En la documentación de API, haga clic en **AddTicket**. Usará esta documentación en el siguiente paso.

Nota: Es posible que algunas funciones de API REST no estén disponibles en la versión actual de Packet Tracer.

Paso 2: Cree una nueva solicitud POST.

- Después de revisar la documentación del método de API de REST **AddTicket**, abra Postman. En el área **Inicio**, haga clic en el signo más para crear una nueva **solicitud sin título**.
- Haga clic en la flecha hacia abajo y cambie el tipo de **GET** a **POST**.
- Introduzca la dirección URL **http://localhost:58000/api/v1/ticket**.
- Debajo del campo URL, haga clic en **Body**. Cambie el tipo a **raw**.
- Haga clic en la flecha abajo junto a **Texto** y cámbielo a **JSON**. Este cambio también establecerá el encabezado HTTP «Content-type» en «application/json» que se requiere para esta llamada a la API.
- Pegue el siguiente objeto JSON en el campo **Body**. Asegúrese de que su código esté correctamente formateado

```
{
  "Username": "cisco",
  «Password»: «cisco123!«
```

```
}
```

Paso 3: Enviar la solicitud POST

- Haga clic en **Enviar** para enviar la solicitud POST al **PT-Controller0**.

Debería obtener una respuesta similar a la siguiente. Sin embargo, **Your_serviceTicket** será un valor real.

```
{
  "response": {
    «IdleTimeout»: 900,
    «ServiceTicket»: "Your_serviceTicket«,
    «SessionTimeout»: 3600
  },
  "version": "1.0"
}
```

- Copie el valor **ServiceTicket** sin las comillas en un archivo de texto para su uso posterior.

Parte 4: Enviar solicitudes REST con Postman

En esta parte, usará su ticket de servicio para enviar tres solicitudes REST al PT-controller0.

Paso 1: Cree una nueva solicitud GET para todos los dispositivos de red de la red.

- En Postman, haga clic en el signo más para crear una nueva **solicitud sin título**.
- Introduzca la dirección URL **http://localhost:58000/api/v1/network-device**.
- Debajo del campo URL, haga clic en **Headers**.
- En la última **clave**, haga clic en el campo **Key** e introduzca **X-Auth-Token**.
- En el campo **Value**, introduzca el valor de su ticket de servicio.

Paso 2: Envíe la solicitud GET.

Haga clic en **Enviar** para enviar la solicitud GET al **PT-Controller0**.

Debe obtener una respuesta que enumera los detalles que tiene el controlador para los nueve dispositivos de red en la red. La respuesta para el primer dispositivo se muestra aquí.

```
{
  "response": [
    {
      «CollectionStatus»: «Managed»,
      «ConnectedInterfaceName»: [
        "GigabitEthernet0/0/0",
        "GigabitEthernet0",
        "FastEthernet0"
      ],
      «ConnectedNetworkDeviceIPAddress»: [
        "192.168.101.1",
        "192.168.101.254",
        "192.168.101.100"
      ],
      «ConnectedNetworkDeviceName»: [
        "R1",

```

```
        «NetworkController»,
        «Example Server»
    ],
    «ErrorDescription»: «»,
    «GlobalCredentialID»: «53046ecc-88c3-49f6-9626-ca8ab9db6725»,
    "hostname": "SWL1",
    «id»: «cat1010BT47-UUID»,
    «InterfaceCount»: «29»,
    «InventoryStatusDetail»: «Gestionado»,
    «LastUpdateTime»: «6»,
    «LastUpdated»: «2020-06-11 22:55:51 »,
    «MacAddress»: «000C.CF42.2B11»,
    «ManagementIpAddress»: «192.168.101.2»,
    «Platform»: «3650»,
    «ProductID»: «3650-24PS»,
    «RecoverabilityFailureReason»: «»,
    «RecoverabilityStatus»: «Accesible»,
    «SerialNumber»: «CAT1010BT47-»,
    «SoftwareVersion»: «16.3.2»,
    «type»: «MultilayerSwitch»,
    «Uptime»: «4 horas, 55 minutos, 11 segundos»
    },
    <output omitted>
    ],
    "version": "1.0"
}
```

Paso 3: Duplicar la solicitud GET y modificarla para todos los hosts de la red.

- En Postman, haga clic con el botón derecho en la pestaña de su solicitud **GET** host y elija **Duplicate Tab**.
- Toda la información en el ticket es la misma excepto la URL. Simplemente cambie el **dispositivo de red** al **host**:

http://localhost:58000/api/v1/host.

Paso 4: Envíe la solicitud GET.

Haga clic en **Send** para enviar la solicitud GET al **PT-Controller0**.

Debe obtener una respuesta que enumera los detalles que tiene el controlador para los seis dispositivos host de la red. La respuesta para el primer dispositivo se muestra aquí.

```
{
  "response": [
    {
      «ConnectedApmacAddress»: «»,
      «ConnectedApName»: «»,
      «ConnectedInterfaceName»: «GigabitEthernet1/0/24»,
      «ConnectedNetworkDeviceIpAddress»: «192.168.102.2»,
      «ConnectedNetworkDeviceName»: «SWL2»,
      «HostIP»: «192.168.102.3»,
      «HostMac»: «00E0.F96C.155B»,

```

```

        «HostName»: «PC4»,
        «HostType»: «Pc»,
        «id»: «ptt08108Mo8-UUID»,
        «LastUpdated»: «2020-06-11 22:49:32 »,
        «PingStatus»: «ÉXITO»
    },
    <output omitted>
  ],
  "version": "1.0"
}

```

Paso 5: Cierre Postman para liberar memoria en la máquina virtual de DEVASC.

Parte 5: Enviar solicitudes REST con código VS

En esta parte, usará la secuencia de comandos de Python en VS Code para enviar las mismas solicitudes de API que envió en Postman. Sin embargo, también usará Python para bucles, para analizar el JSON y mostrar solo pares de valores de clave específicos.

Paso 1: Utilice un script para solicitar un ticket de servicio.

- a. Abrir VS Code.
- b. Haga clic en **File > Open folder...** y navegue al directorio **devnet-src/ptna**.
- c. Haga clic en **Ok**.

Observe en el panel **EXPLORAR** de la izquierda que se muestran tres scripts: **01_get-ticket.py**, **02_get-network-device.py** y **03_get-host.py**. Revise el código de cada uno. Observe que los scripts para dispositivos de red y hosts requieren que reemplace el valor **Your_ServiceTicket** por el valor que le dio Packet Tracer cuando solicitó un ticket. Solicite un nuevo ticket de servicio para ver la función del script **01_get-ticket.py**.

- d. Abra una ventana de terminal en VS Código: Terminal > Nueva Terminal.
- e. Ejecute **01_get-ticket.py** para ver resultados similares a los siguientes.

```

devasc @labvm: ~/labs/devnet-src/ptna$ python3 01_get-ticket.py
Estado de la solicitud de entradas: 201
El número de ticket de servicio es: Your_serviceTicket
devasc@labvm:~/labs/devnet-src/ptna$

```

- f. Reemplace el valor **Your_ServiceTicket** en **02_get-network-device.py** y **03_get-host.py** con el valor que le dio Packet Tracer.

Paso 2: Utilice una secuencia de comandos para solicitar una lista de dispositivos de red.

Anteriormente en Postman, la llamada a la API del dispositivo de red devolvió una lista de los nueve dispositivos de red y toda la información disponible para cada dispositivo. Sin embargo, el script **02_get-network-device.py** imprime sólo los valores de las claves en las que el programador está interesado: **hostname**, **Platform** y **ManagementAddress**.

En la ventana de terminal, ejecute el script **02_get-network-device.py**.

```

devasc@labvm:~/labs/devnet-src/ptna$ python3 02_get-network-device.py
Request Status: 200
SWL1 3650 192.168.101.2
R1 ISR4300 192.168.1.2
R3 ISR4300 192.168.2.1

```

```
SWR1 3650 10.0.1.2
SWR2 3650 10.0.1.3
R2 ISR4300 192.168.2.2
SWL2 3650 192.168.102.2
SWR4 3650 10.0.1.5
SWR3 3650 10.0.1.4
devasc@labvm:~/labs/devnet-src/ptna$
```

Paso 3: Utilice una secuencia de comandos para solicitar una lista de dispositivos host.

Del mismo modo, el programador optó por enumerar información específica para cada uno de los seis dispositivos host conectados a la red.

En la ventana de terminal, ejecute el script `03_get-host.py`.

```
devasc@labvm:~/labs/devnet-src/ptna$ python3 03_get-host.py
Request Status: 200
PC4 192.168.102.3 00E0.F96C.155B GigabitEthernet1/0/24
PC3 10.0.2.129 0004.9A42.C245 GigabitEthernet1/0/24
PC1 10.0.1.129 00E0.A330.3359 GigabitEthernet1/0/22
PC2 10.0.2.130 0060.47C1.A4DB GigabitEthernet1/0/23
Admin 10.0.1.130 0050.0FCE.B095 GigabitEthernet1/0/21
Example server 192.168.101.100 000A.413D.D793 GigabitEthernet1/0/3
devasc@labvm:~/labs/devnet-src/ptna$
```

Parte 6: Enviar solicitudes REST dentro del Packet Tracer (opcional)

En esta parte, usará los mismos scripts con una pequeña edición para enviar las mismas solicitudes de API dentro de Packet Tracer que envió desde VS Code.

Paso 1: Crear un proyecto en Packet Tracer

- En Packet Tracer, haga clic en **Admin PC**.
- Haga clic en la ficha **Programming** (Programación).
- Actualmente no hay ningún proyecto. Haga clic en **New**.
- Introduzca **API REST** como nombre y elija **Empty - Python** como plantilla.
- Haga clic en **Create**.

El proyecto **API REST (Python)** se crea ahora con una secuencia de comandos **main.py** en blanco.

Paso 2: Modifique los scripts para que se ejecuten dentro de Packet Tracer.

El acceso desde una aplicación a otra en el mismo equipo host requiere que el número de puerto se especifique en la URL. Sin embargo, Packet Tracer está simulando una red real. En el mundo real, normalmente no especifica el número de puerto al realizar solicitudes de API. Además, usaría un nombre de dominio o una dirección IP en la URL.

- En **VS Code**, copie el código de `03_get-host.py`.
- En la pestaña **Admin > Programming**, haga doble clic en el script `main.py` para abrirlo.
- Pegue el código en el script `main.py`.
- Cambie el `api_url`. Reemplace `localhost: 58000/api/v1/host` con `192.168.101.254/api/v1/host`.

- e. Las ediciones se guardan automáticamente. Haga clic en **Run**. La salida de Packet Tracer no simula exactamente lo que ve en la línea de comandos de Linux. Sin embargo, debería ver resultados similares como se muestra a continuación.

```
Iniciando API REST (Python)...
('Request status: ', 200)
('PC4', '\ t', ' 192.168.102.3 ', '\ t', ' 00E0.F96C.155B', '\ t', ' GigabitetherNet1/0/24 ')
('PC3', '\ t', ' 10.0.2.129 ', '\ t', ' 0004.9A42.C245', '\ t', ' GigabitetherNet1/0/24 ')
('PC1', '\ t', ' 10.0.1.129 ', '\ t', ' 00E0.A330.3359', '\ t', ' GigabitetherNet1/0/22 ')
('PC2', '\ t', ' 10.0.2.130 ', '\ t', ' 0060.47C1.A4DB', '\ t', ' GigabitetherNet1/0/23 ')
('Admin', '\ t', ' 10.0.1.130 ', '\ t', ' 0050.0FCE.B095', '\ t', ' GigabitetherNet1/0/21 ')
('Example Server', '\ t', ' 192.168.101.100 ', '\ t', ' 000A.413D.D793', '\ t', ' GigabitetherNet1/0/3 ')
REST APIs (Python) finished running.
```

- f. Copie y pegue **02_get-network-device.py** en **main.py**. Cambie la URL y ejecútelo.

```
REST APIs (Python) finished running.
Starting REST API (Python)...
('Request status: ', 200)
('SWL1', '\ t', ' 3650 ', '\ t', '192.168.101.2')
('R1', '\ t', ' ISR4300 ', '\ t', '192.168.1.2')
('R3', '\ t', ' ISR4300 ', '\ t', '192.168.2.1')
('SWR1', '\ t', ' 3650 ', '\ t', '10.0.1.2')
('SWR2', '\ t', ' 3650 ', '\ t', '10.0.1.3')
('R2', '\ t', ' ISR4300 ', '\ t', '192.168.2.2')
('SWL2', '\ t', ' 3650 ', '\ t', '192.168.102.2')
('SWR4', '\ t', ' 3650 ', '\ t', '10.0.1.5')
('SWR3', '\ t', ' 3650 ', '\ t', '10.0.1.4')
REST APIs (Python) finished running.
```